



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-06: Arreglos y Listas
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero B
Alumnos participantes:	Guanotoa Escobar Karla Leonela
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. Rubén Caiza
Link de GitHub:	https://github.com/Karla-ge08/Ape06_Creaci-n_LecturayEscritura_Ficheros

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Conocer en manejo de arreglos y listas que permiten la manipulación de grandes cantidades de datos.

Específicos:

- Implementar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre arreglos estáticos (1D y 2D).
- Desarrollar estructuras de datos dinámicas (Listas enlazadas, ArrayList) para asignación flexible de memoria en tiempo de ejecución.
- Aplicar algoritmos de ordenamiento (QuickSort, MergeSort) y búsqueda (Binaria) para optimizar la complejidad temporal ($O(n \log n)$, $O(\log n)$) en grandes volúmenes de datos.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas.
- Papel
- Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:



- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): Visual Studio Core, Code Blocks, Apache NetBeans, PSeInt Word.

2.6 Actividades por desarrollar.

1) Enunciado ejercicio 1.

Crear un ArrayList con 5 nombres de estudiantes.

2) Análisis del problema.

- **Variables de entrada:** 5 cadenas de texto (nombres).
- **Proceso:** Instanciar estructura de datos dinámica (ArrayList en Java, vector en C++). Insertar 5 registros de tipo cadena. Iterar estructura.
- **Salida:** Impresión por consola de los 5 nombres de estudiantes.

3) Algoritmo.

1. Inicio.
2. Inicializar estructura de datos.
3. Insertar "Ana".
4. Insertar "Luis".
5. Insertar "Carlos".
6. Insertar "Marta".
7. Insertar "Pedro".
8. Recorrer estructura e imprimir elementos.
9. Fin.

4) Pseudocódigo.

Proceso Ejercicio1

// Definición de variables y simulación de estructura dinámica

Definir nombres Como Caracter;

Dimension nombres[5]; // Se dimensiona a 5 para este caso específico

// Asignación de datos (Simulando el add/push_back)

nombres[1] <- "Ana";

nombres[2] <- "Luis";

nombres[3] <- "Carlos";

nombres[4] <- "Marta";

nombres[5] <- "Pedro";

// Recorrido de la estructura para mostrar los datos

Para i <- 1 Hasta 5 Con Paso 1 Hacer

Escribir "Estudiante [", i, "]: ", nombres[i];

FinPara

FinProceso

5) Diagrama de Flujo.

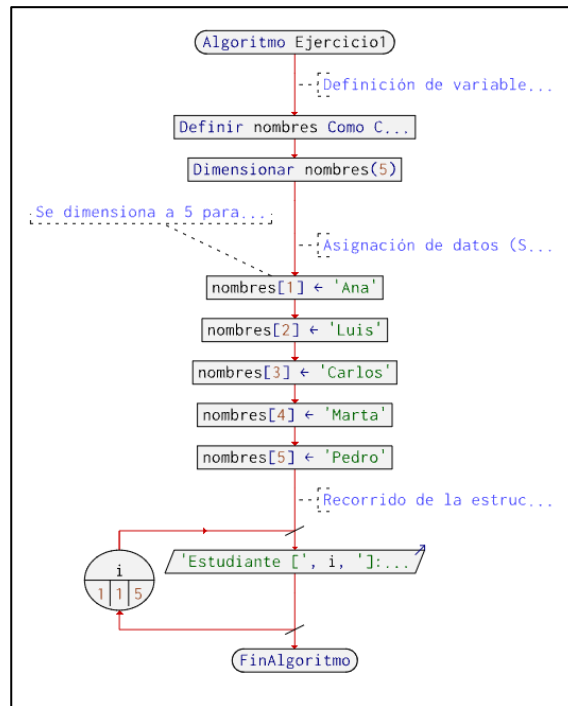


Figura 1 Diagrama de Flujo

6) Códigos en C++.

```
#include <iostream>
#include <vector> // Librería necesaria para usar arreglos dinámicos en C++
#include <string>
```

```
using namespace std;
```

```
int main() {
    // 1. Declaración de la estructura dinámica (vector de strings)
    vector<string> nombres;

    // 2. Inserción de datos usando el método push_back()
    nombres.push_back("Ana");
    nombres.push_back("Luis");
    nombres.push_back("Carlos");
    nombres.push_back("Marta");
    nombres.push_back("Pedro");

    // 3. Recorrido del vector utilizando su tamaño dinámico (size)
    cout << "--- LISTA DE ESTUDIANTES ---" << endl;
    for (size_t i = 0; i < nombres.size(); i++) {
        cout << "Estudiante [" << i << "]: " << nombres[i] << endl;
    }

    return 0;
}
```

7) Códigos en Java.

```
import java.util.ArrayList; // Importación de la clase ArrayList
public class ejercicio1ape6 {
    public static void main(String[] args) {
```



```
// 1. Declaración e instanciación del ArrayList tipado con String
ArrayList<String> nombres = new ArrayList<>();

// 2. Inserción de datos mediante el método add()
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Carlos");
nombres.add("Marta");
nombres.add("Pedro");

// 3. Recorrido del ArrayList usando un bucle for clásico y el método size()
System.out.println("--- LISTA DE ESTUDIANTES ---");
for (int i = 0; i < nombres.size(); i++) {
    // Se utiliza get(i) para acceder al elemento en la posición i
    System.out.println("Estudiante [" + i + "]: " + nombres.get(i));
}
}
```

8) Pruebas de solución.

```
"C:\Users\USUARIO\Desktop\  X  +  v
--- LISTA DE ESTUDIANTES ---
Estudiante [0]: Ana
Estudiante [1]: Luis
Estudiante [2]: Carlos
Estudiante [3]: Marta
Estudiante [4]: Pedro

Process returned 0 (0x0)   execution time : 1.121 s
Press any key to continue.
|
```

Figura 2 Prueba de solución C++

```
--- LISTA DE ESTUDIANTES ---
Estudiante [0]: Ana
Estudiante [1]: Luis
Estudiante [2]: Carlos
Estudiante [3]: Marta
Estudiante [4]: Pedro

BUILD SUCCESS

Total time: 2.357 s
Finished at: 2026-05-30T15:09:41-05:00
```

Figura 3 Prueba de solución JAVA

1) Enunciado ejercicio 2.

Crear un ArrayList con 6 notas.

2) Análisis del problema.



- **Variables de entrada:** 6 valores numéricos decimales (double/float).
- **Proceso:** Crear una lista dinámica para números. Agregar 6 calificaciones. Calcular un promedio opcional (buena práctica para validar tipos numéricos) y mostrar las notas.
- **Salida:** Listado de las 6 notas en consola.

3) Algoritmo.

1. Inicio.
2. Inicializar lista dinámica de tipo numérico.
3. Añadir 6 notas (ej. 14.5, 18.0, etc.).
4. Iterar la lista e imprimir cada nota.
5. Fin.

4) Pseudocódigo.

Proceso Ejercicio2

// Declaración para almacenar datos numéricos con decimales
Definir notas Como Real;
Dimension notas[6];

// Asignación de 6 calificaciones
notas[1] <- 15.5;
notas[2] <- 18.0;
notas[3] <- 12.75;
notas[4] <- 19.5;
notas[5] <- 14.0;
notas[6] <- 16.25;

// Bucle para mostrar el contenido de la estructura
Escribir "--- REGISTRO DE NOTAS ---";
Para i<-1 Hasta 6 Con Paso 1 Hacer
Escribir "Nota ", i, ": ", notas[i];
FinPara

FinProceso

5) Diagrama de Flujo.

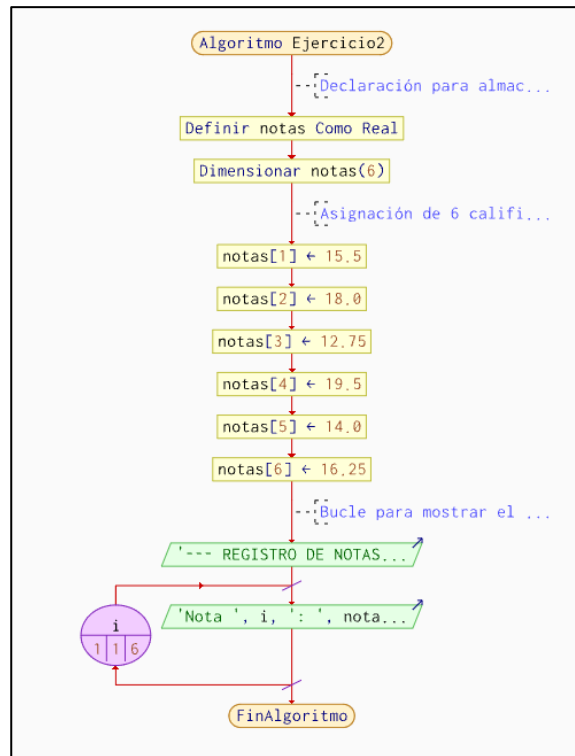


Figura 4 Diagrama de Flujo

6) Código en C++.

```
#include <iostream>
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    // 1. Declaración del vector para valores con punto flotante
    vector<double> notas;
```

```
    // 2. Inserción de 6 notas
```

```
    notas.push_back(15.5);
    notas.push_back(18.0);
    notas.push_back(12.75);
    notas.push_back(19.5);
    notas.push_back(14.0);
    notas.push_back(16.25);
```

```
    // 3. Iteración e impresión de resultados
```

```
    cout << "--- REGISTRO DE NOTAS ---" << endl;
    for (size_t i = 0; i < notas.size(); i++) {
        cout << "Nota " << i + 1 << ": " << notas[i] << endl;
    }
```

```
    return 0;
```

```
}
```

7) Código en Java.

```
import java.util.ArrayList;
public class ejercicio2ape6 {
```



```
public static void main(String[] args) {  
    // 1. Uso de la clase Wrapper 'Double' para colecciones de primitivos  
    ArrayList<Double> notas = new ArrayList<>();  
  
    // 2. Agregando los 6 elementos al ArrayList  
    notas.add(15.5);  
    notas.add(18.0);  
    notas.add(12.75);  
    notas.add(19.5);  
    notas.add(14.0);  
    notas.add(16.25);  
  
    // 3. Impresión mediante bucle foreach (otra forma de iterar)  
    System.out.println("--- REGISTRO DE NOTAS ---");  
    int contador = 1; // Variable auxiliar para enumerar  
    for (Double nota : notas) {  
        System.out.println("Nota " + contador + ": " + nota);  
        contador++;  
    }  
}
```

8) Pruebas de solución.

```
"C:\Users\USUARIO\Desktop\  
--- REGISTRO DE NOTAS ---  
Nota 1: 15.5  
Nota 2: 18  
Nota 3: 12.75  
Nota 4: 19.5  
Nota 5: 14  
Nota 6: 16.25  
  
Process returned 0 (0x0)   execution time : 1.045 s  
Press any key to continue.  
|
```

Figura 5 Prueba de solución C++



```
--- REGISTRO DE NOTAS ---  
Nota 1: 15.5  
Nota 2: 18.0  
Nota 3: 12.75  
Nota 4: 19.5  
Nota 5: 14.0  
Nota 6: 16.25  
  
-----  
BUILD SUCCESS  
  
-----  
Total time: 1.622 s  
Finished at: 2026-05-30T16:12:04-05:00  
  
-----
```

Figura 6 Prueba de solución JAVA

1) Enunciado ejercicio 3.

Registrar productos y realizar búsquedas.

2) Análisis del problema.

- **Variables de entrada:** Cadena productoBuscado. Elementos predefinidos en la lista.
- **Proceso:** Crear lista de productos, solicitar un producto a buscar, usar un bucle para comparar (== o .equals()) o métodos integrados (contains()).
- **Salida:** Mensaje booleano o de texto confirmando si el producto se encuentra en stock.

3) Algoritmo.

1. Inicio.
2. Llenar lista con productos (ej. "Laptop", "Mouse", "Teclado").
3. Leer el producto que el usuario desea buscar.
4. Buscar en la estructura de datos.
5. Si existe, imprimir "Producto encontrado". Si no, "Producto no encontrado".
6. Fin.

4) Pseudocódigo.

Proceso Ejercicio3

*Definir inventario Como Caracter;
Definir buscar, productoBuscado Como Caracter;
Definir encontrado Como Logico;
Dimension inventario[3];*

*// Carga de inventario base
inventario[1] <- "Laptop";
inventario[2] <- "Mouse";
inventario[3] <- "Teclado";*

*// Entrada de búsqueda
Escribir "Ingrese producto a buscar:";
Leer productoBuscado;*



encontrado <- Falso;

// Algoritmo de búsqueda lineal

Para i<-1 Hasta 3 Con Paso 1 Hacer

Si Mayusculas(inventario[i]) = Mayusculas(productoBuscado) Entonces

encontrado <- Verdadero;

FinSi

FinPara

// Salida de resultados

Si encontrado = Verdadero Entonces

Escribir "El producto SÍ está en el inventario.";

SiNo

Escribir "El producto NO fue encontrado.";

FinSi

FinProceso

5) Diagrama de Flujo.

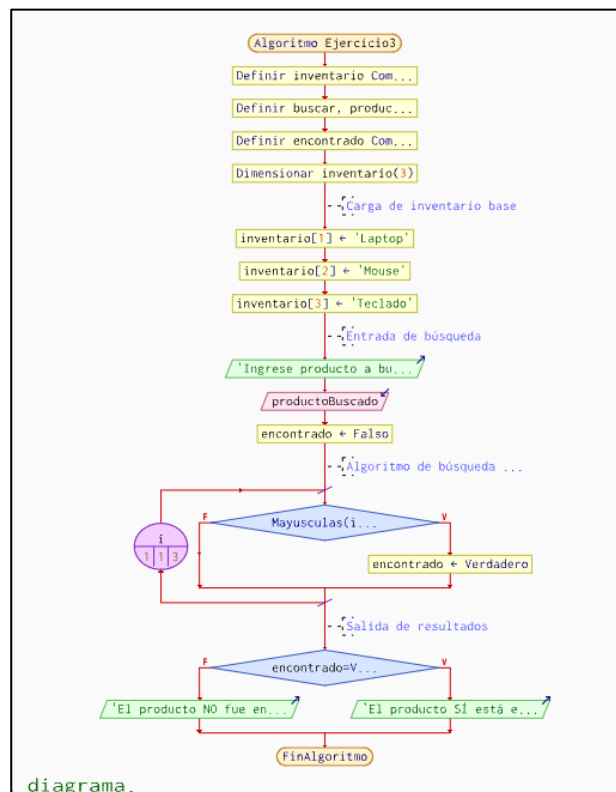


Figura 7 Diagrama de Flujo

6) Código en C++.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <algorithm> // Librería para funciones como 'find'
```

```
using namespace std;
```

```
int main() {
```

```
    // 1. Declaración e inicialización rápida
```

```
    vector<string> productos = {"Laptop", "Mouse", "Teclado", "Monitor"};
```



```
string buscado;
```

```
// 2. Captura de datos del usuario  
cout << "Ingrese el producto a buscar: ";  
cin >> buscado;
```

```
// 3. Búsqueda utilizando iteradores de la STL (Standard Template Library)  
auto it = find(productos.begin(), productos.end(), buscado);
```

```
// 4. Validación del resultado  
if (it != productos.end()) {  
    cout << "Resultado: El producto " << buscado << " SI se encuentra en  
stock." << endl;  
} else {  
    cout << "Resultado: Producto NO encontrado." << endl;  
}
```

```
return 0;
```

```
}
```

7) Código en Java.

```
import java.util.ArrayList;  
import java.util.Scanner;
```

```
public class ejercicio3ape6 {  
    public static void main(String[] args) {  
        ArrayList<String> productos = new ArrayList<>();  
        Scanner scanner = new Scanner(System.in); // Objeto para lectura de teclado
```

```
// 1. Población de datos  
productos.add("Laptop");  
productos.add("Mouse");  
productos.add("Teclado");  
productos.add("Monitor");
```

```
// 2. Interacción con el usuario  
System.out.print("Ingrese el producto a buscar: ");  
String buscado = scanner.nextLine();
```

```
// 3. Verificación rápida usando el método nativo contains()  
if (productos.contains(buscado)) {  
    System.out.println("Resultado: El producto " + buscado + " Sí se  
encuentra en stock.");  
} else {  
    System.out.println("Resultado: Producto NO encontrado.");  
}
```

```
scanner.close(); // Liberación del recurso
```

```
}
```

```
}
```

8) Pruebas de solución.



```
"C:\Users\USUARIO\Desktop\ X + v
Ingrese el producto a buscar: Laptop
Resultado: El producto 'Laptop' SI se encuentra en stock.

Process returned 0 (0x0) execution time : 3.454 s
Press any key to continue.
|
```

Figura 8 Prueba de solución C++

```
Ingrese el producto a buscar: parlantes
Resultado: Producto NO encontrado.

-----
BUILD SUCCESS
-----

Total time: 15.379 s
Finished at: 2026-05-30T16:23:32-05:00
-----
|
```

Figura 9 Prueba de solución en JAVA

1) Enunciado ejercicio 4.

Crear un menú CRUD usando ArrayList

2) Análisis del problema.

- **Variables de entrada:** Opciones del menú (int), Valores a insertar/modificar/borrar (String).
- **Proceso:** Un ciclo do-while o while con un switch-case. Operaciones: C (add/push_back), R (iterar/imprimir), U (set/sobreescribir en índice), D (remove/erase).
- **Salida:** Menú interactivo y reportes de la lista tras cada modificación.

3) Algoritmo.

1. Inicio.
2. Iniciar lista vacía.
3. Mostrar Menú: 1. Crear, 2. Leer, 3. Actualizar, 4. Eliminar, 5. Salir.
4. Leer opción.
5. Ejecutar acción correspondiente según la opción seleccionada.
6. Repetir hasta que la opción sea 5.
7. Fin.

4) Pseudocódigo.

Proceso MenuCRUD

Definir opcion Como Entero;

// PSeInt no soporta arreglos dinámicos reales, se asume un máximo y un contador

Definir contador Como Entero;

contador <- 0;



Repetir

Escribir "--- MENÚ CRUD ---";
Escribir "1. Agregar elemento";
Escribir "2. Mostrar elementos";
Escribir "3. Salir"; // Simplificado para pseudocódigo
Leer opcion;

Segun opcion Hacer

1:
Escribir "Lógica de Inserción aquí...";
contador <- contador + 1;
2:
Escribir "Lógica de Lectura aquí...";
3:
Escribir "Saliendo...";
De Otro Modo:
Escribir "Opción inválida.";

FinSegun

Hasta Que opcion = 3

FinProceso

5) Diagrama de Flujo.

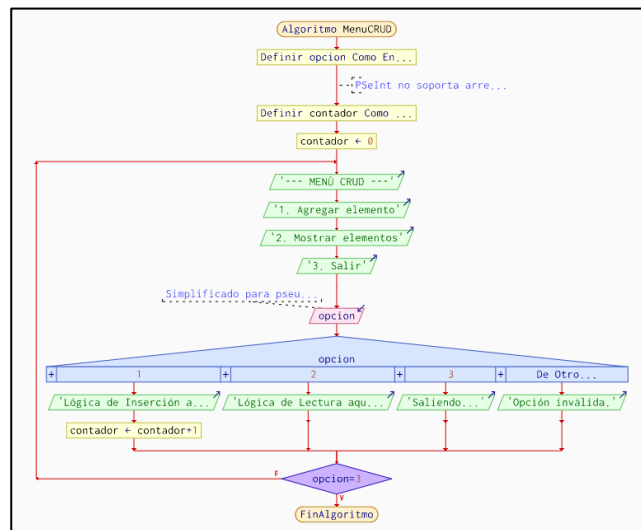


Figura 10 Diagrama de Flujo

6) Código en C++.

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

int main() {
    vector<string> items;
    int opcion, indice;
    string dato;

    do {
        // Renderizado del Menú
        cout << "\n--- MENU CRUD ---" << endl;
```



```
cout << "1. Crear (Add)\n2. Leer (Read)\n3. Actualizar (Update)\n4. Eliminar\n(Delete)\n5. Salir" << endl;\ncout << "Seleccione: ";\ncin >> opcion;\n\nswitch (opcion) {\n    case 1: // CREATE\n        cout << "Ingrese nuevo item: ";\ncin >> dato;\n        items.push_back(dato); // [cite: 26]\n        cout << "Guardado." << endl;\n        break;\n    case 2: // READ\n        cout << "- Lista de Items -" << endl;\n        for (size_t i = 0; i < items.size(); i++) {\n            cout << "[" << i << "]" << items[i] << endl; // [cite: 26]\n        }\n        break;\n    case 3: // UPDATE\n        cout << "Indice a actualizar: ";\ncin >> indice;\n        if(indice >= 0 && indice < items.size()) {\n            cout << "Nuevo valor: ";\ncin >> dato;\n            items[indice] = dato; // Sobreescritura directa\n            cout << "Actualizado." << endl;\n        }\n        break;\n    case 4: // DELETE\n        cout << "Indice a eliminar: ";\ncin >> indice;\n        if(indice >= 0 && indice < items.size()) {\n            items.erase(items.begin() + indice); // [cite: 26]\n            cout << "Eliminado." << endl;\n        }\n        break;\n    case 5:\n        cout << "Saliendo..." << endl;\n        break;\n}\n} while (opcion != 5);\n\nreturn 0;\n}
```

7) Código en Java.

```
import java.util.ArrayList;\nimport java.util.Scanner;\npublic class ejercicio4ape6 {\n\n    public static void main(String[] args) {\n        ArrayList<String> items = new ArrayList<>();\n        Scanner scan = new Scanner(System.in);\n        int opcion = 0;
```



```
do {
    System.out.println("\n--- MENU CRUD ---");
    System.out.println("1. Crear\n2. Leer\n3. Actualizar\n4. Eliminar\n5.
Salir");
    System.out.print("Seleccione: ");
    opcion = scan.nextInt();
    scan.nextLine(); // Limpiar el buffer

    switch (opcion) {
        case 1: // CREATE
            System.out.print("Ingrese nuevo item: ");
            items.add(scan.nextLine()); // [cite: 26]
            System.out.println("Guardado.");
            break;
        case 2: // READ
            System.out.println("- Lista de Items -");
            for (int i = 0; i < items.size(); i++) {
                System.out.println "[" + i + " ] " + items.get(i)); // [cite: 26]
            }
            break;
        case 3: // UPDATE
            System.out.print("Indice a actualizar: ");
            int idxU = scan.nextInt();
            scan.nextLine();
            if (idxU >= 0 && idxU < items.size()) {
                System.out.print("Nuevo valor: ");
                items.set(idxU, scan.nextLine()); // set() para actualizar en Java
                System.out.println("Actualizado.");
            }
            break;
        case 4: // DELETE
            System.out.print("Indice a eliminar: ");
            int idxD = scan.nextInt();
            if (idxD >= 0 && idxD < items.size()) {
                items.remove(idxD); // [cite: 26]
                System.out.println("Eliminado.");
            }
            break;
    }
} while (opcion != 5);
scan.close();
}
```

8) Pruebas de solución.



```
--- MENU CRUD ---
1. Crear (Add)
2. Leer (Read)
3. Actualizar (Update)
4. Eliminar (Delete)
5. Salir
Seleccione: 3
Indice a actualizar: 0
Nuevo valor: 2
Actualizado.

--- MENU CRUD ---
1. Crear (Add)
2. Leer (Read)
3. Actualizar (Update)
4. Eliminar (Delete)
5. Salir
Seleccione: 2
- Lista de Items -
[0] 2

--- MENU CRUD ---
1. Crear (Add)
2. Leer (Read)
3. Actualizar (Update)
4. Eliminar (Delete)
5. Salir
Seleccione: 3
Indice a actualizar: 0
Nuevo valor: Computador
Actualizado.

--- MENU CRUD ---
1. Crear (Add)
2. Leer (Read)
3. Actualizar (Update)
4. Eliminar (Delete)
5. Salir
Seleccione: 2
- Lista de Items -
[0] Computador
```

Figura 11 Prueba de solución C++



```
--- MENU CRUD ---
1. Crear
2. Leer
3. Actualizar
4. Eliminar
5. Salir
Seleccione: 2
- Lista de Items -
[0] teclado

--- MENU CRUD ---
1. Crear
2. Leer
3. Actualizar
4. Eliminar
5. Salir
Seleccione: 3
Indice a actualizar: 0
Nuevo valor: Pantalla
Actualizado.

--- MENU CRUD ---
1. Crear
2. Leer
3. Actualizar
4. Eliminar
5. Salir
Seleccione: 4
Indice a eliminar: 0
Eliminado.
```

Figura 12 Prueba de solución de JAVA

1) Enunciado ejercicio 5.

Registrar nombres y notas.

2) Análisis del problema.

- **Variables de entrada:** Cadenas para nombres, decimales para notas.
- **Proceso:** Manejar la correlación entre un nombre y su calificación. Usaremos dos arreglos paralelos (uno de Strings y otro de Doubles) que comparten el mismo índice para relacionar los datos.
- **Salida:** Un reporte en tabla que muestre Nombre - Nota.

3) Algoritmo.

1. Inicio.
2. Crear ListaNombres y ListaNotas.
3. Pedir la cantidad de estudiantes a registrar.
4. Bucle para ingresar el nombre e ingresarle su nota.
5. Bucle para imprimir ambas listas usando un índice común (i).
6. Fin.

4) Pseudocódigo.



Proceso Ejercicio5

*Definir nombres Como Caracter;
Definir notas Como Real;
Dimension nombres[2]; // Ajustable según necesidad
Dimension notas[2];*

// Inserción paralela

*Para i<-1 Hasta 2 Con Paso 1 Hacer
 Escribir "Nombre del estudiante ", i, ":";
 Leer nombres[i];
 Escribir "Nota del estudiante ", i, ":";
 Leer notas[i];
FinPara*

// Lectura paralela (sincronizada por el índice i)

*Escribir "--- REGISTRO DE NOTAS ---";
Para i<-1 Hasta 2 Con Paso 1 Hacer
 Escribir nombres[i], " - Nota: ", notas[i];
FinPara*

FinProceso

5) Diagrama de Flujo.

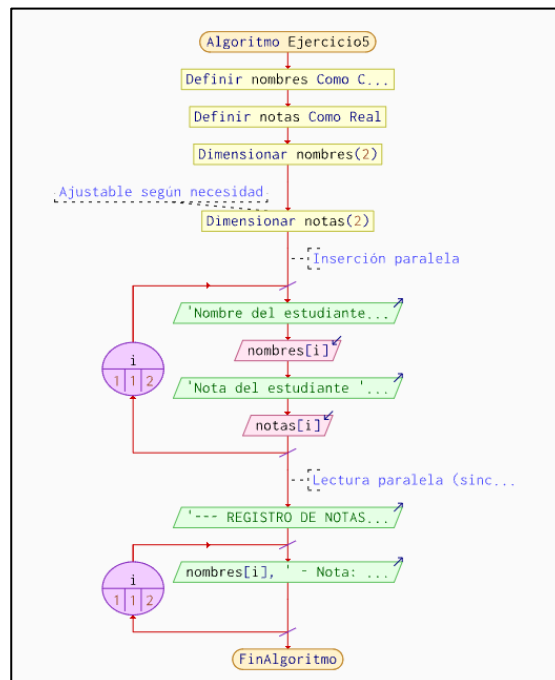


Figura 13 Diagrama de Flujo

6) Código en C++.

```
#include <iostream>
#include <vector>
#include <string>
```

```
using namespace std;
```

```
int main() {
    // 1. Declaración de vectores paralelos
    vector<string> nombres;
```



```
vector<double> notas;
string tempNombre;
double tempNota;
int cantidad;

// 2. Solicitar dimensionamiento lógico
cout << "Cuántos estudiantes registrará?: ";
cin >> cantidad;

// 3. Captura y vinculación de datos
for(int i = 0; i < cantidad; i++) {
    cout << "Nombre [" << i+1 << "]: ";
    cin >> tempNombre;
    nombres.push_back(tempNombre);

    cout << "Nota [" << i+1 << "]: ";
    cin >> tempNota;
    notas.push_back(tempNota);
}

// 4. Reporte final sincronizando el índice 'i'
cout << "\n--- REPORTE FINAL ---" << endl;
for(size_t i = 0; i < nombres.size(); i++) {
    cout << nombres[i] << "\t | \t" << notas[i] << endl;
}

return 0;
}
```

7) Código en Java.

8) Pruebas de solución.

```
"C:\Users\USUARIO\Desktop\" X + v
Cuántos estudiantes registrará?: 4
Nombre [1]: Joel
Nota [1]: 10
Nombre [2]: Leonela
Nota [2]: 9
Nombre [3]: Mayte
Nota [3]: 5.2
Nombre [4]: Rocio
Nota [4]: 7.9

--- REPORTE FINAL ---
Joel | 10
Leonela | 9
Mayte | 5.2
Rocio | 7.9

Process returned 0 (0x0) execution time : 37.788 s
Press any key to continue.
|
```

Figura 14 Prueba de solución C++



```
Cuantos registros desea ingresar?: 2
Ingrese nombre: Joel
Ingrese nota de Joel: 2,5
Ingrese nombre: Karla
Ingrese nota de Karla: 10

--- BOLETÍN DE CALIFICACIONES ---
Estudiante: Joel -> Calificación: 2.5
Estudiante: Karla -> Calificación: 10.0

-----
BUILD SUCCESS
-----

Total time: 20.425 s
Finished at: 2026-05-30T19:02:23-05:00
-----
```

Figura 15 Prueba de solución JAVA

1) Enunciado del ejercicio 6.

Crear un sistema de inventario

2) Análisis del problema.

- **Variables de entrada:** Nombre del producto (Cadena), Cantidad (Entero), Precio Unitario (Real).
- **Proceso:** Implementar listas paralelas para almacenar las características del inventario. Calcular el valor total por producto (cantidad * precio) y un gran total del inventario.
- **Salida:** Reporte tabular con los datos del inventario y la suma total valorizada.

3) Algoritmo.

1. Inicio.
2. Inicializar tres listas dinámicas paralelas (Nombres, Cantidades, Precios).
3. Ingresar los datos de varios productos.
4. Recorrer las listas, calculando el subtotal por ítem.
5. Acumular el valor total.
6. Imprimir el inventario y el total.
7. Fin.

4) Pseudocódigo.

Proceso Ejercicio6_Inventario

Definir nombres Como Caracter;

Definir cantidades Como Entero;

Definir precios, totalInventario, subtotal Como Real;

Dimension nombres[2], cantidades[2], precios[2]; // Simulación de listas

totalInventario <- 0;

// Inserción de datos

Para i <- 1 Hasta 2 Hacer

Escribir "Ingrese nombre del producto: "; Leer nombres[i];

Escribir "Ingrese cantidad: "; Leer cantidades[i];

Escribir "Ingrese precio: "; Leer precios[i];

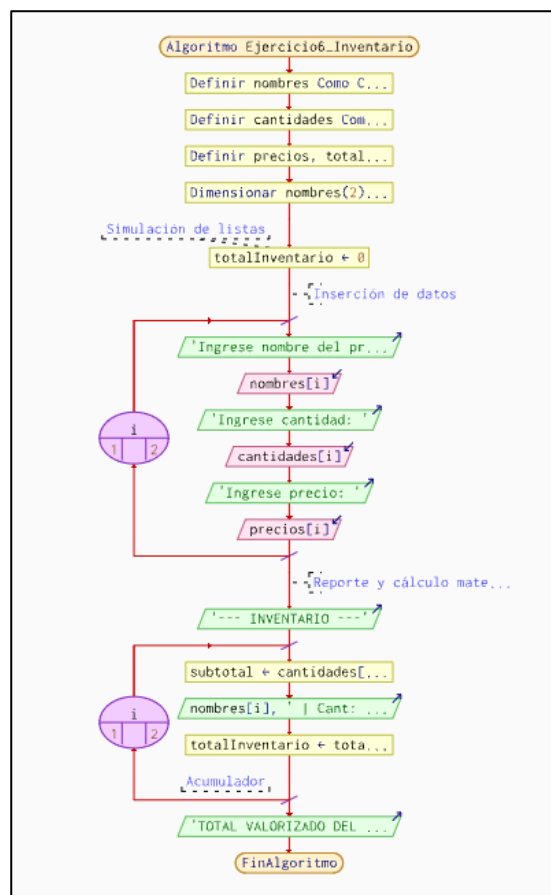
FinPara



```
// Reporte y cálculo matemático
Escribir "--- INVENTARIO ---";
Para i<-1 Hasta 2 Hacer
    subtotal <- cantidades[i] * precios[i];
    Escribir nombres[i], " | Cant: ", cantidades[i], " | Precio: $", precios[i], " |
Subtotal: $", subtotal;
    totalInventario <- totalInventario + subtotal; // Acumulador
FinPara
Escribir "TOTAL VALORIZADO DEL INVENTARIO: $", totalInventario;

FinProceso
```

5) Diagrama de Flujo.



6) Código en C++.

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

int main() {
    // 1. Declaración de vectores paralelos para el inventario
    vector<string> nombres = {"Disco Duro", "Memoria RAM"};
    vector<int> cantidades = {10, 25};
    vector<double> precios = {55.50, 32.00};
    double totalInventario = 0;

    cout << "--- SISTEMA DE INVENTARIO ---" << endl;
```



```
// 2. Procesamiento de los datos almacenados
for(size_t i = 0; i < nombres.size(); i++) {
    double subtotal = cantidades[i] * precios[i]; // Cálculo por ítem
    totalInventario += subtotal;

    cout << nombres[i] << "\t| Cant: " << cantidades[i]
        << " | Precio: $" << precios[i] << " | Subtotal: $" << subtotal << endl;
}

// 3. Salida del gran total
cout << "-----" << endl;
cout << "Valor Total en Inventario: $" << totalInventario << endl;

return 0;
}
```

7) Código en Java.

```
import java.util.ArrayList;
public class ejercicio6ape6 {

    public static void main(String[] args) {
        // 1. Instanciación de colecciones tipadas
        ArrayList<String> nombres = new ArrayList<>();
        ArrayList<Integer> cantidades = new ArrayList<>();
        ArrayList<Double> precios = new ArrayList<>();

        // 2. Población del inventario (hardcodeado para demostración)
        nombres.add("Teclado Mecanico"); cantidades.add(15); precios.add(45.0);
        nombres.add("Monitor 24"); cantidades.add(5); precios.add(120.5);

        double totalGlobal = 0;

        System.out.println("--- SISTEMA DE INVENTARIO ---");

        // 3. Iteración y cálculos
        for(int i = 0; i < nombres.size(); i++) {
            double subtotal = cantidades.get(i) * precios.get(i);
            totalGlobal += subtotal; // Sumatoria

            System.out.println(nombres.get(i) + " | Cant: " + cantidades.get(i) +
                " | Precio: $" + precios.get(i) + " | Subtotal: $" + subtotal);
        }
        System.out.println("-----");
        System.out.println("Valor Total en Inventario: $" + totalGlobal);
    }
}
```

8) Pruebas de solución.



```
"C:\Users\USUARIO\Desktop\ X + v
--- SISTEMA DE INVENTARIO ---
Disco Duro      | Cant: 10 | Precio: $55.5 | Subtotal: $555
Memoria RAM     | Cant: 25 | Precio: $32   | Subtotal: $800
-----
Valor Total en Inventario: $1355

Process returned 0 (0x0)   execution time : 1.046 s
Press any key to continue.
|
```

Figura 16 Prueba de solución C++

```
--- SISTEMA DE INVENTARIO ---
Teclado Mecanico | Cant: 15 | Precio: $45.0 | Subtotal: $675.0
Monitor 24''    | Cant: 5  | Precio: $120.5 | Subtotal: $602.5
-----
Valor Total en Inventario: $1277.5

BUILD SUCCESS

Total time: 1.643 s
Finished at: 2026-05-31T22:15:42-05:00
-----
```

Figura 17 Prueba de solución JAVA

1) Enunciado del ejercicio 7.

Comparar arreglos y ArrayList.

2) Análisis del problema.

- **Variables de entrada:** Ninguna (demostración teórica/práctica).
- **Proceso:** Crear un arreglo estático y una lista dinámica. Observar cómo se declaran y cómo se determina su tamaño.
- **Salida:** Impresión de elementos demostrando que la estructura estática requiere un tamaño fijo en la compilación, mientras que la dinámica no.

3) Algoritmo.

1. Inicio.
2. Declarar arreglo estático de tamaño 3.
3. Declarar Lista dinámica vacía.
4. Asignar valores a ambos.
5. Imprimir tamaño y capacidad de ambos.
6. Fin.

4) Pseudocódigo.

Proceso Ejercicio7_Comparacion

// En PSeInt todo son arreglos estáticos (Dimension),

// se explicará mediante mensajes la diferencia conceptual.

Escribir "--- Arreglo Estático ---";



Escribir "Su tamaño se define al inicio y no puede cambiar."
Dimensionar arregloFijo[3];
arregloFijo[1] <- 10;

Escribir "--- Lista Dinámica (ArrayList/Vector) ---";
Escribir "Su tamaño crece o se encoge en tiempo de ejecución."
Escribir "No requiere definir un límite inicial.";

FinProceso

5) Diagrama de Flujo.

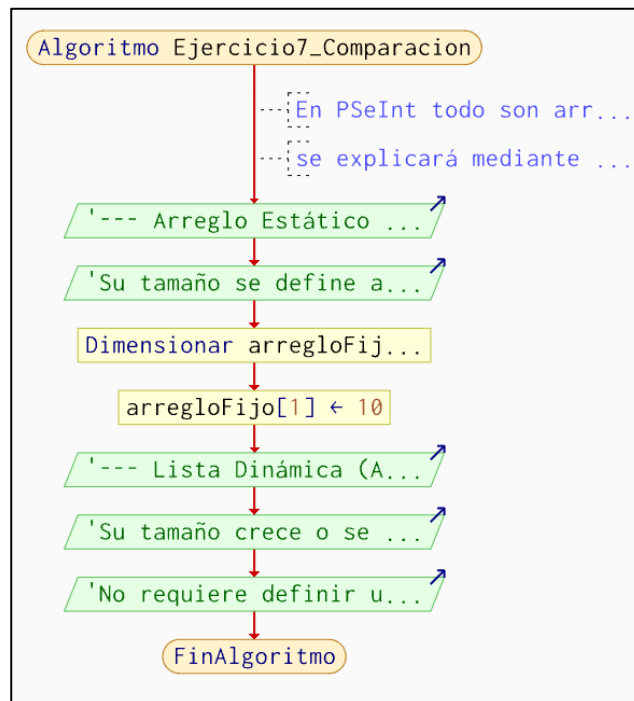


Figura 18 Diagrama de flujo

6) Código en C++.

```
#include <iostream>
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    // 1. Arreglo Primitivo (Estático)
```

```
    int arreglo[3] = {10, 20, 30}; // Tamaño fijo definido aquí
```

```
    // 2. Arreglo Dinámico (Vector)
```

```
    vector<int> listaDinamica; // Empieza vacío (tamaño 0)
```

```
    listaDinamica.push_back(10);
```

```
    listaDinamica.push_back(20);
```

```
    cout << "--- COMPARACION ---" << endl;
```

```
    // El arreglo no tiene método .size() propiamente, se calcula con sizeof
```

```
    cout << "Tamano Arreglo Estatico: " << sizeof(arreglo)/sizeof(arreglo[0]) << endl;
```

```
    // El vector sabe su tamaño exactamente
```

```
    cout << "Tamano Vector Dinamico: " << listaDinamica.size() << endl;
```



```
return 0;
```

```
}
```

7) Código en Java.

```
import java.util.ArrayList;
public class ejercicio7ape6 {

    public static void main(String[] args) {
        // 1. Arreglo nativo (Estático)
        int[] arreglo = new int[3];
        arreglo[0] = 10;

        // 2. ArrayList (Dinámico)
        ArrayList<Integer> listaDinamica = new ArrayList<>();
        listaDinamica.add(10); // Crece automáticamente

        System.out.println("--- COMPARACION ---");
        // En arreglos usamos la propiedad 'length'
        System.out.println("Capacidad maxima del arreglo: " + arreglo.length);

        // En ArrayList usamos el método 'size()'
        System.out.println("Tamaño actual del ArrayList: " + listaDinamica.size());
    }
}
```

8) Pruebas de solución.

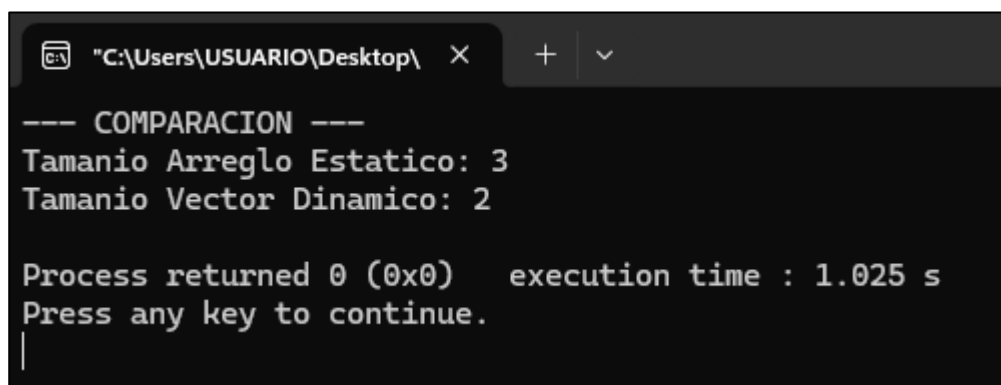


Figura 19 Prueba de solución C++

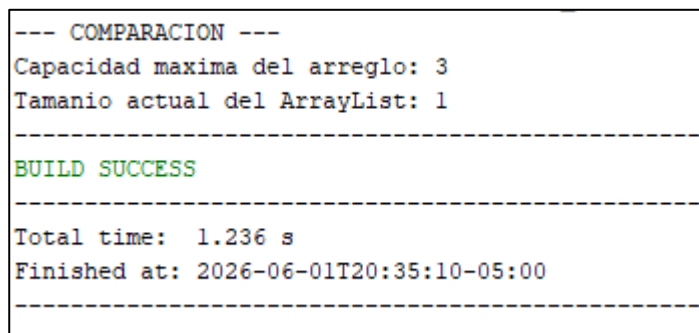


Figura 20 Prueba de solución JAVA

1) Enunciado del ejercicio 8.

Comparar inserción de datos.



2) Análisis del problema.

- **Variables de entrada:** Nuevos datos a insertar.
- **Proceso:** Intentar agregar un dato a un arreglo lleno (lo cual causa error o desbordamiento) vs agregar un dato a una lista dinámica usando `push_back()` o `add()`.
- **Salida:** Demostración de flexibilidad en la inserción.

3) Algoritmo.

1. Inicio.
2. Definir arreglo estático y llenarlo.
3. Intentar agregar un elemento extra (se demostrará que no es posible directamente sin crear uno nuevo).
4. Definir ArrayList. Agregar elementos usando su método de inserción secuencial.
5. Fin.

4) Pseudocódigo.

Proceso Ejercicio8_Insercion

Dimensionar estatico[2];

estatico[1] <- 5; estatico[1] <- 10;

// Si intentamos estatico[2] <- 15; -> ERROR FUERA DE LIMITES

Escribir "Arreglo lleno, no se pueden agregar más elementos sin re-dimensionar.";

// Simulación ArrayList

Escribir "En una lista dinámica, usamos metodos como add() o push_back() para insertar sin preocuparnos del límite.";

FinProceso

5) Diagrama de Flujo.

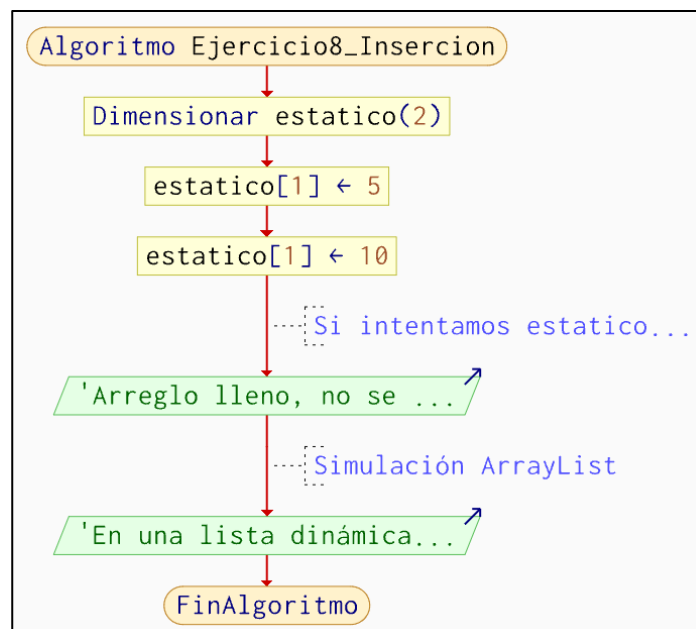


Figura 21 Diagrama de flujo



6) Código en C++.

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int arr[2] = {1, 2};
    // arr[2] = 3; // PELIGRO: Comportamiento indefinido, desbordamiento de
    memoria.

    vector<int> vec;
    vec.push_back(1); //
    vec.push_back(2);

    cout << "Insertando en vector dinamico..." << endl;
    // Podemos seguir agregando sin romper la memoria
    vec.push_back(3);
    vec.push_back(4);

    for(int val : vec) {
        cout << "Valor: " << val << endl;
    }

    return 0;
}
```

7) Código en Java.

```
import java.util.ArrayList;
public class ejercicio8ape6 {

    public static void main(String[] args) {
        int[] arr = new int[2];
        arr[0] = 1; arr[1] = 2;
        // arr[2] = 3; // Lanza ArrayIndexOutOfBoundsException

        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(1);
        lista.add(2);

        System.out.println("Insertando en ArrayList de forma segura...");
        // Crece de forma transparente para el programador
        lista.add(3);
        lista.add(4);

        System.out.println("Contenido: " + lista);
    }
}
```

8) Pruebas de solución.



```
"C:\Users\USUARIO\Desktop\ x + v
Insertando en vector dinamico...
Valor: 1
Valor: 2
Valor: 3
Valor: 4

Process returned 0 (0x0)   execution time : 1.023 s
Press any key to continue.
|
```

Figura 22 Prueba de solución C++

```
Insertando en ArrayList de forma segura...
Contenido: [1, 2, 3, 4]
-----
BUILD SUCCESS
-----
Total time: 1.406 s
Finished at: 2026-06-01T20:42:41-05:00
-----
```

Figura 23 Prueba de solución JAVA

1) Enunciado del ejercicio 9.

Manipular turnos con LinkedList.

2) Análisis del problema.

- **Variables de entrada:** Nombres de pacientes/clientes.
- **Proceso:** Implementar una estructura basada en nodos (LinkedList / list). Simular una cola FIFO (First In, First Out).
- **Salida:** Turnos agregados y el orden de atención (eliminación del principio).

3) Algoritmo.

1. Inicio.
2. Instanciar una LinkedList.
3. Agregar "Turno 1 - Juan", "Turno 2 - María".
4. Imprimir la lista.
5. Atender (remove) el primer elemento.
6. Mostrar la lista actualizada.
7. Fin.

4) Pseudocódigo.

Proceso Ejercicio9_LinkedList

Escribir "--- GESTION DE TURNOS ---";
Escribir "Llega: Juan (Turno 1)";
Escribir "Llega: Maria (Turno 2)";
Escribir "Lista actual: Juan, Maria";



Escribir "Llamando al siguiente...";
Escribir "Atendiendo a: Juan (Se elimina del frente de la lista)";
Escribir "Lista actual: Maria";

FinProceso

5) Diagrama de Flujo.

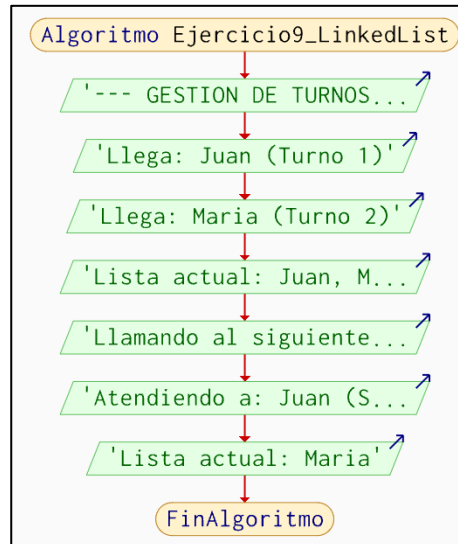


Figura 24 Diagrama de flujo

6) Código en C++.

```
#include <iostream>
#include <list> // Librería específica para listas doblemente enlazadas
#include <string>
```

```
using namespace std;
```

```
int main() {
    list<string> turnos;

    // Agregar turnos al final de la cola
    turnos.push_back("T01 - Juan"); //
    turnos.push_back("T02 - Maria");
    turnos.push_back("T03 - Carlos");

    cout << "--- COLA DE ATENCION ---" << endl;
    for(string turno : turnos) cout << turno << endl;

    // Atender (Eliminar el primer elemento)
    cout << "\nAtendiendo a: " << turnos.front() << endl;
    turnos.pop_front(); // Remueve de manera eficiente el nodo inicial

    cout << "\n--- COLA ACTUALIZADA ---" << endl;
    for(string turno : turnos) cout << turno << endl;

    return 0;
}
```

7) Código en Java.

```
import java.util.LinkedList; //
```



```
public class ejercicio9ape6 {  
  
    public static void main(String[] args) {  
        // En Java, LinkedList implementa las interfaces List y Deque  
        LinkedList<String> turnos = new LinkedList<>();  
  
        // Método para agregar al final  
        turnos.addLast("T01 - Juan");  
        turnos.addLast("T02 - Maria");  
        turnos.addLast("T03 - Carlos");  
  
        System.out.println("--- COLA DE ATENCION ---");  
        System.out.println(turnos);  
  
        // Método para obtener y remover el primer nodo  
        String atendido = turnos.removeFirst();  
        System.out.println("\nAtendiendo a: " + atendido);  
  
        System.out.println("\n--- COLA ACTUALIZADA ---");  
        System.out.println(turnos);  
    }  
}
```

8) Pruebas de solución.

Figura 25 Prueba de solución C++



```
--- COLA DE ATENCION ---  
[T01 - Juan, T02 - Maria, T03 - Carlos]  
  
Atendiendo a: T01 - Juan  
  
--- COLA ACTUALIZADA ---  
[T02 - Maria, T03 - Carlos]  
-----  
BUILD SUCCESS  
-----  
Total time: 1.316 s  
Finished at: 2026-06-01T20:57:28-05:00  
-----
```

Figura 26 Prueba de solución JAVA

1) Enunciado del ejercicio 10.

Simular una fila de atención.

2) Análisis del problema.

- **Variables de entrada:** Opciones de un menú interactivo. Nombres de clientes.
- **Proceso:** Crear un bucle (menú) que permita al usuario decidir si "Llega un cliente" o "Se atiende un cliente", gestionando en tiempo real la LinkedList.
- **Salida:** Mensajes dinámicos de quién llegó, quién fue atendido y el estado de la fila.

3) Algoritmo.

1. Inicio.
2. Crear LinkedList vacía.
3. Mostrar menú: 1. Ingresar cliente, 2. Atender, 3. Salir.
4. Si 1: Pedir nombre y agregar al final.
5. Si 2: Verificar si hay clientes. Si hay, remover del frente e imprimir. Si no, avisar que está vacía.
6. Repetir hasta que elija 3.
7. Fin.

4) Pseudocódigo.

Proceso Ejercicio10_Fila

Definir opcion Como Entero;

Repetir

Escribir "1. Llega Cliente | 2. Atender | 3. Salir";

Leer opcion;

Segun opcion Hacer

1: Escribir "Cliente encolado.";

2: Escribir "Cliente despachado (sale del frente).";

FinSegun

Hasta Que opcion = 3

FinProceso



5) Diagrama de Flujo.

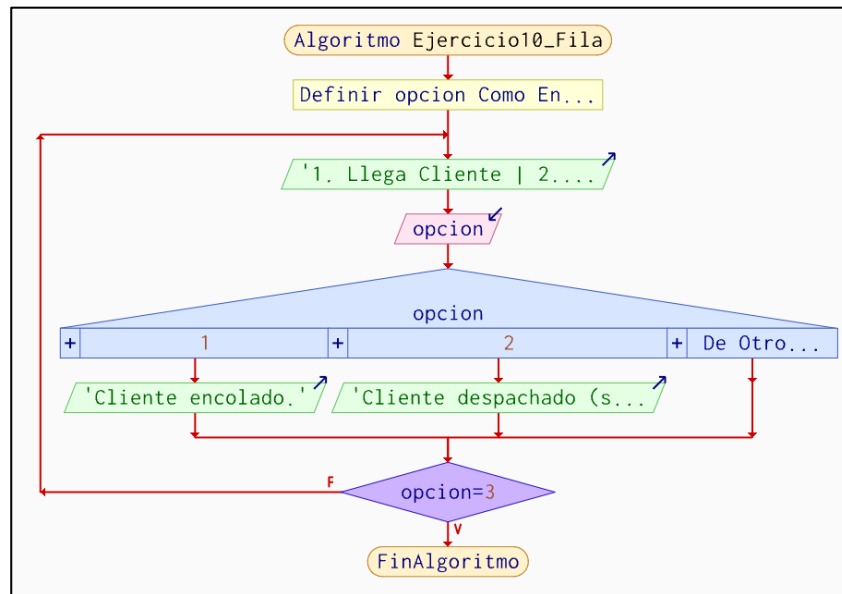


Figura 27 Diagrama de flujo

6) Código en C++.

```
#include <iostream>
```

```
#include <list>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    list<string> fila;
```

```
    int opcion;
```

```
    string cliente;
```

```
    do {
```

```
        cout << "\n--- FILA DEL BANCO ---" << endl;
```

```
        cout << "1. Llega cliente\n2. Atender\n3. Salir\nOpcion: ";
```

```
        cin >> opcion;
```

```
        switch(opcion) {
```

```
            case 1:
```

```
                cout << "Nombre del cliente: ";
```

```
                cin >> cliente;
```

```
                fila.push_back(cliente);
```

```
                cout << "-> " << cliente << " se unio a la fila." << endl;
```

```
                break;
```

```
            case 2:
```

```
                if(!fila.empty()) {
```

```
                    cout << "-> Atendiendo a: " << fila.front() << endl;
```

```
                    fila.pop_front();
```

```
                } else {
```

```
                    cout << "-> La fila esta vacia!" << endl;
```

```
                }
```

```
                break;
```

```
        }
```



```
} while(opcion != 3);
```

```
return 0;
```

```
}
```

7) Código en Java.

```
import java.util.LinkedList;
import java.util.Scanner;
public class ejercicio10ape6 {

    public static void main(String[] args) {
        LinkedList<String> fila = new LinkedList<>();
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\n--- FILA DEL BANCO ---");
            System.out.println("1. Llega cliente\n2. Atender\n3. Salir\nOpcion: ");
            opcion = sc.nextInt();
            sc.nextLine(); // Limpiar buffer

            switch(opcion) {
                case 1:
                    System.out.print("Nombre del cliente: ");
                    String cliente = sc.nextLine();
                    fila.addLast(cliente);
                    System.out.println("-> " + cliente + " se unio a la fila.");
                    break;
                case 2:
                    // Verificación de seguridad para evitar Excepciones
                    if(!fila.isEmpty()) {
                        System.out.println("-> Atendiendo a: " + fila.removeFirst());
                    } else {
                        System.out.println("-> ¡La fila esta vacia!");
                    }
                    break;
            }
        } while(opcion != 3);
        sc.close();
    }
}
```

8) Pruebas de solución.



```
"C:\Users\USUARIO\Desktop\ x + v

--- FILA DEL BANCO ---
1. Llega cliente
2. Atender
3. Salir
Opcion: 1
Nombre del cliente: karla
-> karla se unio a la fila.

--- FILA DEL BANCO ---
1. Llega cliente
2. Atender
3. Salir
Opcion: 2
-> Atendiendo a: karla

--- FILA DEL BANCO ---
1. Llega cliente
2. Atender
3. Salir
Opcion: 3

Process returned 0 (0x0)   execution time : 12.419 s
Press any key to continue.
|
```

Figura 28 Prueba de solución C++

```
--- FILA DEL BANCO ---
1. Llega cliente
2. Atender
3. Salir
Opcion:
1
Nombre del cliente: Abisag
-> Abisag se unio a la fila.

--- FILA DEL BANCO ---
1. Llega cliente
2. Atender
3. Salir
Opcion:
2
-> Atendiendo a: Abisag

--- FILA DEL BANCO ---
1. Llega cliente
2. Atender
3. Salir
Opcion:
3

-----
BUILD SUCCESS
-----

Total time: 14.118 s
Finished at: 2026-06-01T21:28:40-05:00
-----
```

Figura 29 Prueba de solución JAVA

2.7 Resultados obtenidos

Conoce en manejo de arreglos y listas que permiten la manipulación de grandes cantidades de datos.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo



- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Los arreglos estáticos limitan la escalabilidad de las aplicaciones debido a su asignación de memoria fija en tiempo de compilación.
- Las estructuras dinámicas previenen la fragmentación y optimizan el uso de memoria RAM al escalar según la carga de información.
- La eficiencia de las consultas sobre conjuntos masivos depende estrictamente de la complejidad temporal de los algoritmos de ordenamiento aplicados.

2.10 Recomendaciones

- Implementar ArrayList (Java) o std::vector (C++) como estándar frente a los arreglos primitivos para registros de longitud variable.
- Integrar clases de flujos de salida (ej. BufferedWriter) para volcar el contenido de las listas en archivos .txt y garantizar la persistencia de los registros.
- Ejecutar MergeSort o QuickSort antes de operaciones de lectura intensivas para habilitar métodos de Búsqueda Binaria de forma obligatoria.